

Received 18 May 2025, accepted 16 June 2025, date of publication 18 June 2025, date of current version 25 June 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3581139

RESEARCH ARTICLE

A GPT-Based Code Review System With Accurate Feedback for Programming Education

DONG-KYU LEE^{ID} AND INWHEE JOE^{ID}

Department of Computer Science, Hanyang University, Seoul 04763, Republic of Korea

Corresponding author: Inwhee Joe (iwjoe@hanyang.ac.kr)

This work was supported in part by DLAB in RECRUITING PRIMARY AND SECONDARY students for the usability testing of the Code Review System and by C3 Coding in providing an online judge system dataset for experimental evaluation.

ABSTRACT The increasing demand for programming education and growing class sizes require immediate and personalized feedback. However, integrating Large Language Models (LLMs) like ChatGPT in introductory programming courses raises concerns about AI-assisted cheating. In large-scale settings, faulty code submissions may lead LLMs to overanalyze, causing unnecessary token consumption. This paper proposes a GPT-4o-based code review system that provides accurate feedback while reducing token usage and preventing AI-assisted cheating. Unlike general-purpose LLM tools for professionals, the system is pedagogically designed for primary and secondary students by focusing on review necessity and learner-friendly feedback. The system features a Code Review Module (CRM) that reduces token usage via a Review Necessity Chain (RNC), and Code Correctness Check Module (CCM) combining test case validation with LLM-based assessment. To prevent AI-assisted cheating, the system provides automated feedback on submitted code without prompting and revealing correct answers, which are accessed only through the “Ask Code Tutor” button. In usability test, the system detected up to 42.86% more errors than a conventional online judge. BERTScore analysis showed that over 80% of the system-generated reviews were semantically aligned with human feedback. A performance comparison with state-of-the-art systems demonstrated a blocking success rate of 86%, with a comparable review omission rate. These results indicate that the system provides more accurate feedback than conventional automated code reviews, while achieving token efficiency and supporting self-directed learning through educational feedback. Thus, it can serve as a practical solution for scalable programming education in primary and secondary classes.

INDEX TERMS Large language models (LLMs), GPT-4o, programming education, learner-friendly code reviews, LangChain.

I. INTRODUCTION

Digital Transformation across all sectors of society is currently having a positive impact on various socio-economic areas. At this point of digital transformation, a global trend is forming that emphasizes the importance of education to equip everyone with digital competencies so that they can participate in and benefit evenly from the digital world [1]. According to the Organization for Economic Cooperation

and Development Skill Outlook 2019 report [2], digital competencies will be closely related to all jobs in the future, which highlights the need to cultivate comprehensive digital skills throughout life.

In response to this global trend, the United States has adopted a collaborative approach to advancing AI literacy in education, with the federal government setting a national strategy and state governments implementing initiatives in coordination. The U.S. Department of Education outlined key priorities such as educational equity, personalized learning, and support for educators in its 2023 report, Artificial

The associate editor coordinating the review of this manuscript and approving it for publication was Yu-Da Lin^{ID}.

Intelligence and the Future of Teaching and Learning [3]. The U.S Department of Education recently disclosed a detailed inventory of 56 active AI use cases for AI integration into education [4]. These commitments to AI integration reflect a policy approach grounded in both innovation and operational effectiveness across the educational system.

In Finland, the Ministry of Education and Culture has outlined a comprehensive national strategy in its report, Policies for the Digitalisation of Education and Training until 2027 [5]. The report presents a vision of Finland to become the world's leading country in sustainable digitalisation in education by 2027 and emphasizes the promotion of equity, accessibility, and lifelong learning. The strategy highlights the importance of equipping all learners and educators with high-level digital competencies, fostering the development of user-driven digital solutions, and building connected digital environments that support effective teaching methods and help make decisions based on data.

In Singapore, the Ministry of Education has presented EdTech Masterplan 2030 [6], which envisions “technology-transformed learning” to prepare students for a technology-driven world. The strategy emphasizes the use of AI-enabled platforms such as the Singapore Student Learning Space (SLS) to provide personalized learning, enhance feedback through adaptive tools, and strengthen digital and data literacy for both students and teachers. The plan of Singapore also incorporates e-pedagogy, cyber wellness, and collaborative innovation as core components of the digital learning ecosystem.

In line with these global efforts, educational institutions in Korea have also enhanced education in order to foster digital AI literacy through the national curriculum of 2022 revision from Ministry of Education. Particularly in primary and secondary schools, computer and information education is being expanded by increasing either computer and information class or discretionary time [7]. While computer and information education has become mandatory in primary and secondary schools, educational institutions can allow the introduction of various coding courses tailored to students' career paths and aptitudes. Primary schools can now organize and operate Practical Science for previously 17 hours, but now over 34 hours. Secondary schools can organize and operate computer and information subject for previously 34 hours, but now over 68 hours. In addition, most core subjects such as Korean language, science, social studies, and art are increasingly related with contents that promote digital competencies.

Meanwhile, structured programming assessments have been practiced in various countries. For example, in the United Kingdom, the British Informatics Olympiad (BIO) provides another form of structured programming assessment [8]. The BIO consists of two stages: a first round administered at schools and a final round held at a university.

The first round is a three-hour in-school exam where students solve algorithmic problems by writing programs in any language and submitting their code along with written

responses. Teachers evaluate the submissions manually based on provided mark schemes and submit the scores electronically. Unlike Online platforms that provide automated test case evaluation, this method relies entirely on teacher-led assessment.

In the United States, the AP Computer Science A exam allows high school students to get college credit through official programming assessment. Administered digitally via the Bluebook app, the exam includes multiple-choice and free-response sections [9]. The free-response section asks students to implement code that meets specific functional requirements using standard programming constructs. Evaluation focuses not only on correctness but also on code structure, resembling the feedback way used by a tutor or peer for code review.

Based on the preceding examples, K-12 learners have many opportunities to access online and offline programming education environments. Accordingly, the demand for programming language education is increasing, and class sizes are also growing. However, due to the nature of programming education, various correct solutions are possible. Since each learner submits different code, providing personalized code reviews can be an effective approach. Traditional code review methods have limitations in providing immediate and personalized feedback [10], [11]. Automated test case methods offered by online systems only detect issues such as compilation errors or input/output mismatches, which may be difficult and not adequate for beginners to interpret [12]. Peer reviews conducted through online forums also require significant time and effort and may be ineffective due to varying learner levels [13]. Tutor reviews, while effective for personalization, are challenging to scale due to offline classroom constraints.

The emergence of GPT, a generative pretrained transformer model, offers an effective solution to overcome the limitations of traditional code review methods [13], [14]. Codex, integrated into GPT, can translate natural language prompts into code, supporting programming education effectively. These capabilities make GPT well-suited for immediate code reviews and personalized feedback.

However, the use of LLMs in educational settings raises concerns, particularly regarding AI-assisted cheating when students directly use generative tools in introductory programming courses. Recent studies have pointed out serious ethical concerns, emphasizing that the misuse of AI chatbots such as ChatGPT can weaken students' critical thinking skills and reduce their sense of autonomy [15], [16]. Addressing AI-assisted cheating therefore requires not only technical safeguards, but also pedagogical and ethical interventions, achieved through close collaboration between the education and technology sectors [17].

Moreover, LLMs may generate subtly incorrect or context-insensitive code, especially when handling incomplete or poorly structured student submissions [14], [18]. In large-scale educational contexts, this can lead to not only ineffective feedback but also unnecessary token consumption

due to the overanalysis of such invalid inputs. Therefore, it is necessary to develop systems that provide accurate and efficient code reviews while minimizing these risks.

This paper presents a GPT-4o-based code review system designed to support programming education for primary and secondary students. Unlike general-purpose LLM tools such as GitHub Copilot or CodeAid, which focus on developer productivity and skill assessment, the proposed system aims to minimize AI-assisted cheating and reduce unnecessary token usage, while providing learner-friendly feedback.

To support these objectives, the system generates review comments only for student-submitted code without directly providing solutions. Prompts were designed to ensure feedback quality across five dimensions: precision, usefulness, specificity, supportive tone, and learning effect. Students can use the system by selecting problems from an exercise bank, submitting their code, and receiving code correctness checks along with feedback in an integrated learning environment.

The system was developed using learning logs from K-12 learners collected through existing online judges, allowing it to reflect common error patterns and learning behaviors observed in actual classrooms. To further improve feedback efficiency, it incorporates a Review Necessity Chain (RNC), which filters out trivial or meaningless submissions before generating feedback.

To assess the system's effectiveness, three usability evaluations were conducted, involving instructors with over two years of programming education experience as well as elementary and secondary school students. Based on the first and second evaluation results, the system was improved. The following research questions were addressed to validate the enhanced system:

- **RQ1:** Does the Code Correctness Check Module improve the relative error detection rate compared to a traditional online judge system?
- **RQ2:** How semantically similar is the feedback generated by the Code Review Module (CRM) to those produced by human instructors?
- **RQ3:** Does the pipeline of the CRM effectively reduce unnecessary reviews while retaining essential feedback?
- **RQ4:** Can the proposed code review system improve learning efficiency and reduce instructors' feedback workload in large-scale educational settings?

Although global research on AI integration into programming education is becoming increasingly active, South Korea remains underrepresented in this domain—ranking outside the top 20 contributing nations [19]. By presenting a Korea-based case of system development, deployment, and empirical evaluation, this study seeks to contribute to narrowing this geographical and practical gap.

II. RELATED WORK

A. LLMS IN EDUCATIONAL FEEDBACK

As LLMs become widely used in practical applications, educational experts are exploring the potential of using GPT to generate educational feedback on student assignments.

Dai et al. [20] collected business scenario-related data science project proposals from students at an Australian university. The proposals were required to include a project description and business model. Instructors evaluated the submissions based on five criteria: objectives, topic relevance, business benefit, novelty, and clarity, providing text feedback subsequently. After anonymizing the students' personal information, feedback related to 103 students' proposals was compiled. To generate text feedback on these proposals using ChatGPT, the prompt was designed with properties: project's objectives, relevance to the data science topic, business benefit, novelty/creativity, and overall clarity. Each student's proposal text was added after the prompt and submitted to ChatGPT, which generated the feedback. Interestingly, it was noted that the instructors' feedback was on average 109 characters less than that provided by ChatGPT. While a higher word count does not necessarily equate to more effective feedback, three experts rated each piece of feedback for readability and consistency on a scale from 0 to 4, finding that ChatGPT's feedback was more detailed and readable than that of the instructors.

GPT could potentially reduce some of the instructor's workload while providing useful feedback to students in educational settings.

B. CODE REVIEW AUTOMATION ON BASIC PROGRAMMING TRAINING

Learning tools that provide automated code feedback, such as online judge services, deal with code submitted by students in a test case manner. This method runs the students' programs against test cases and then provides feedback on failed cases. The error message is in English and consists of terms related to compilation errors, making it difficult for beginners to understand the cause of the issue.

To provide automatic feedback to novice learners, Rishabh Singh [21] proposed a method where learning tools require error information from instructors, including solutions and potential corrections. Based on this information, the system can automatically identify the minimal corrections to a student's incorrect code and provide feedback on what went wrong, along with a quantifiable measure of how wrong a submitted code is. The feedback provided includes: 1) the location of the error as a line number, 2) the problematic expression on that line, 3) the sub-expression that needs to be modified, and 4) the newly modified value of the sub-expression. The learning tool applies these rules to the student program, searches for candidate programs that require minimal modification, and provides feedback.

The composition of such feedback and the instructor-provided data are critical factors to consider when designing prompts for an LLM-based code review system.

C. PROMPT TEMPLATE FOR CODE REVIEW AUTOMATION

Compared to pre-trained language models (PLMs), LLMs in code review automation provide a more efficient and

general-purpose approach to complex task such as code review automation. Because LLM already internalizes various domain knowledge, it requires fewer resources for domain-specific prior training and can be flexibly applied to a variety of tasks, contributing greatly to the development of code review automation.

To automate code reviews, Lu et al. [22] proposed a pipeline as a cycle of the code review process that predicts the need for reviews, generates review comments, and sequentially performs code improvement tasks.

The three core tasks are:

- **Review necessity prediction (RNP):** Predicts whether a diff hunk requires review (yes/no).
- **Review comment generation (RCG):** Generates comments for diff hunks from both line-level (Crrer dataset) and method-level (Tufano dataset) perspectives.
- **Code refinement (CR):** Makes minor modifications to improve code quality, often resulting in similar input and output code.

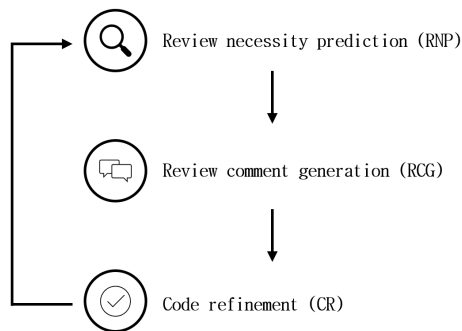


FIGURE 1. The cycle of code review process.

In terms of systematically managing prompt templates in this way, the maintenance task become efficient and effective by fixing the response results of LLMs and updating the prompts in specific areas.

D. LLMS IN COMPUTER SCIENCE CLASS

The recent LLMs, such as the recently released ChatGPT and GitHub Copilot, are receiving significant attention from computer science education experts. Education experts have found that LLMs can generate accurate solutions and accurately explain code for a variety of introductory programming assignments. The outstanding capabilities of these LLMs are generating ongoing discussions about how programming should be taught in computer science classes.

At this point, the opinions of computer science instructors are divided. Zhang et al. [23] interviewed 20 introductory programming instructors about how to apply it to their classes. The author responded that in the short term, many instructors will ban the use of LLMs in class to prevent AI-assisted cheating. On the other hand, the remaining respondents responded that they would be willing to introduce LLMs into their classes. In particular, in the introductory programming class, where students learn programming language proficiency and

algorithms by solving various programming problems, the rules of using ChatGPT, which includes Codex, a powerful code generation model, are needed. If novice learners use ChatGPT to easily generate code through prompts to solve problems, their learning effectiveness may decrease and they may become dependent on GPT.

To prevent AI-assisted cheating side effects that may occur when GPT is introduced into programming language classes, prompts and system flows for generating code review comments need to be designed effectively and systematically. For prompts, Kazemitabaar [11] suggests including the following principles when designing prompts:

- Include at least one input/output example
- Structuring the response, including Markdown and delimiters for display
- Verify the accuracy of submitted code and ensure technical accuracy necessary for code modification
- Use a style and tone that does not make students uncomfortable

When constructing the code feedback system, it is necessary to effectively display the response results generated by GPT through prompts on the web-front by distinguishing between the function of annotating code lines that require modification based on Markdown and learner-friendly feedback.

E. SEMANTIC SIMILARITY ASSESSMENT

BERTScore is a widely used automatic evaluation metric in text generation tasks [24]. Unlike traditional n-gram-based metrics such as BLEU or METEOR, BERTScore employs contextualized embeddings to capture deeper semantic relationships between sentences. Specifically, it computes token-level cosine similarity using BERT-based contextual embeddings, which dynamically reflect the meaning of words based on their surrounding context.

BERTScore has demonstrated stronger correlations with human judgments compared to traditional metrics across various domains [25]. This capability is particularly important in educational settings, where feedback often contains learner-friendly and personalized explanations that cannot be fully captured by exact matching methods.

F. COMPARISON WITH PRIOR GPT-BASED CODE REVIEW SYSTEMS

In recent years, several studies have explored the application of GPT models in automated code feedback systems for education. To situate the proposed system within the landscape of existing research, a comparison is made with three representative GPT-based code review systems.

Nguyen and Allan [14] designed a GPT-4-based system that provides tiered formative feedback to undergraduates. Their system builds on student submissions by suggesting improvements and posing follow-up questions. However, it occasionally builds upon submissions with fundamental flaws (2.65% of cases), highlighting the need for improved correctness filtering.

Crandall et al. [26], [27] developed the ART system, which integrates GPT-3.5-turbo into a DevOps-style feedback loop. Feedback is automatically triggered via GitHub Actions upon each commit. While students found the tone and content supportive, some expressed concerns about receiving detailed feedback every time they pushed code, which could be overwhelming.

Sahai et al. [12] proposed an iterative LLM repair method based on test case results, targeting high school students. Their evaluation of this system showed that GPT-4 outperformed traditional APR tools, repairing 74.9% of buggy submissions. However, due to inconsistencies in the output format, the use of prompt orchestration tools such as LangChain was recommended.

In contrast to these approaches, the proposed system is explicitly designed for primary and secondary students and introduces a Review Necessity Chain (RNC) to reduce unnecessary feedback and API usage. Instead of directly modifying code, it provides annotated comments with supportive, age-appropriate explanations. LangChain is also integrated to ensure prompt consistency, as similarly proposed in prior work. These design choices are intended to reduce operational costs and minimize the risk of AI-assisted cheating, making the system both practical and sustainable in real-world educational settings.

III. METHOD

This study was conducted through a six-step process, as illustrated in Figure 2. This system underwent three iterations of user testing and improvements. First, in the dataset collection phase, test data were extracted from the logs of an existing online judge system (OJ), which automatically evaluates submitted code by running it against predefined test cases, to evaluate the system's performance. Second, during the prompt design phase, the prompt templates were built and refined using the collected dataset. Third, in the system design guideline phase, further improvements were derived by comparing literature and the OJ, leading to the establishment of system guidelines. Fourth, in the initial system development phase, the environment for deployment was prepared, modules were implemented, and the user interface was built. Fifth, usability testing was conducted, and based on the results, improvement strategies were defined and implemented.

Finally, after iterative refinements, the final system's performance was evaluated through the research questions, validating its effectiveness as a learning support tool for primary and secondary school programming education.

A. DATASET COLLECTION

In order to systematically manage test data, the data frame was composed of eight-labels as follows:

- **Exercise ID (Ex.ID):** Primary key of exercise in the online judge system.
- **Title:** Name of exercise.

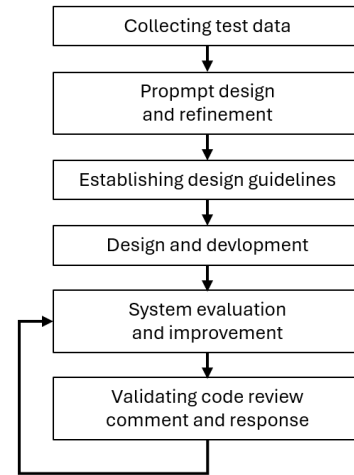


FIGURE 2. Process of system development and evaluation.

- **Description (Desc):** Instructions including exercise requirements, input and output examples.
- **Solution:** Instructor's answer code.
- **Submitted code:** Code submitted by students.
- **Solved subs:** Number of correct answers among student submissions.
- **Total subs:** Total number of submissions.
- **Accuracy:** Proportion of correct answers.

Exercises, student submissions, and instructor solutions were collected from Company C's OJ [28] for an introductory Python programming course, referencing the data collection method in [23].

- **Raw data collection:** Since 2021, Python submissions, exercises, and solutions were secured.
- **Static-based filtering:** Duplicate submissions and comments within code were removed to avoid affecting GPT performance.

A total of 93 test data for 27 questions were entered into the data frame. An example is shown in Table 1.

B. PROMPT TEMPLATE DESIGN AND ENHANCEMENT

To provide code feedback, prompt templates were designed for the Code Review Module (CRM), based on the three-stage feedback generation pipeline proposed by Lu et al [22]. To ensure learner-friendly feedback, the prompts were designed to be technically accurate, age-appropriate. All outputs are formatted in Markdown to ensure clear visibility in both the comment and code editor areas. An example of the prompt structure is presented in Table 2.

- **Role-setting prompt:** Processes learner's requests by setting the role of the model.
- **Review necessity prediction prompt:** Responds with a binary label of "yes" or "no" indicating the need for review of the code submitted by the learner. If "no", end response.
- **Review comment generation prompt:** Generates review comments with specific formatting.

TABLE 1. Example of test data in data frame.

Ex.ID	1229
Title	Function 12
Desc.	[Instruction] Function <code>sum()</code> receives three integers as parameters and returns the total. Function <code>avg()</code> is a function that receives three integers as parameters and returns the average. Let's print the output by calling the functions <code>sum()</code> and <code>avg()</code> respectively. [Test] Input example: 10, 20, 30 Output example: 60, 20.0
Answer code	<pre>def Sum(a, b, c): s = a+b+c return s def avg(a, b, c): s = Sum(a, b, c) avg = s/3 return avg a = int(input()) b = int(input()) c = int(input()) print(Sum(a, b, c)) print(avg(a, b, c))</pre>
Sub. code	<pre>def sum(a,b,c): print(a+b+c) def avg(a,b,c): print((a+b+c)//3) a = int(input()) b = int(input()) c = int(input()) sum(a,b,c) avg(a,b,c)</pre>
Solved	139
Total	310
Acc.	45%

The prompt templates for the Code Correctness Check Module (CCM) are designed to evaluate learners' submitted code by comparing it to the correct answer, compiling the code, and returning the results to the learner.

This module follows a collaborative workflow that not only performs conventional test case validation but also detects code quality issues that may not be identified through mechanical checks alone.

Additional verification is conducted using a strict code check prompt, which further analyzes the submitted code for potential problems. An example prompt is shown in Table 3.

C. SYSTEM DESIGN GUIDELINE ESTABLISHMENT

Design guidelines were established based on literature research and improvements to the online judge (OJ). First, learner-friendly feedback must be provided. In the OJ, only incorrect answers to problems can be checked, and learners had to identify problems themselves through error messages based on test cases.

During this process, learners often experienced difficulties using English and interpreting error messages. By introducing a LLM-based code review system, it is possible to prevent learning deviation and improve the programming learning experience by providing code review comments that are immediate, accurate, and with a supportive tone.

Second, comments must be provided on lines that need to be fixed in the student-submitted code. In the OJ, the type

TABLE 2. Example of prompt templates for the the Code Review Module (CRM).

Role-setting prompt	
Role: Teacher Task: Provide code review of student-submitted [SubmittedCode] Purpose: Help solve [PythonProblem]	
Review necessity prediction prompt	
Instruction	Before reviewing the [SubmittedCode], determine whether a review is necessary. Respond 'yes' if the [SubmittedCode] contains mistakes or improvements are needed. Respond 'no_correct' if the [SubmittedCode] is already correct and matches the [Solution], meaning no review is needed. Respond 'no_meaningless' if the [SubmittedCode] is too simple (e.g., 'print()', single numbers, random characters) and does not attempt to solve the [Python-Problem], meaning no review is needed. Respond with only one of the three options: 'yes', 'no_correct', or 'no_meaningless'.
Review comment generation prompt	
Style&Tone	Vocabulary suitable for primary and secondary school students.
Instruction	Include [Example]/[Example], [RNP]/[RNP], [RC]/[RC], and [R]/[R]. Add 'Code to fix' comment to incorrect lines. Reply to [RC]/[RC] with commented [Code]. Respond with [R]/[R] within three sentences.
Restriction	Do not directly modify or show fixed code in [RC]/[RC] and [R]/[R].
Exercise	[Instruction] Function <code>sum()</code> receives three integers and returns the total. Function <code>avg()</code> returns the average.
Test	Input: 10, 20, 30; Output: 60, 20.0
Submitted Code	<pre>def sum(a,b,c): print(a+b+c) def avg(a,b,c): print((a+b+c)//3) a = int(input()) b = int(input()) c = int(input()) sum(a,b,c) avg(a,b,c)</pre>
Solution	<pre>def Sum(a, b, c): s = a+b+c return s def avg(a, b, c): s = Sum(a, b, c) avg = s/3 return avg a = int(input()) b = int(input()) c = int(input()) print(Sum(a, b, c)) print(avg(a, b, c))</pre>
Example	[RC] length = 42.195 print("Marathon's distance %.fkm" % length) # Code to fix print("Marathon's distance is %.2fkm" % length) [/RC] [R] Good job! But there is a typo in 'distence'. Correct it to 'distance'. [/R]

of error found during the compilation process and the code line where the error occurred were shown in text, making it difficult for learners in the introductory course to understand the feedback.

To solve this inconvenience, when a user clicks the code submission button, a "Code to fix" comment is added to the code line that needs to be fixed through the Code Review Module (CRM). This feature allows learners to immediately identify the code where problems occur.

Third, AI-assisted cheating must be limited. The system only provides comments on the code submitted by the learner and does not directly present the fixed code, thereby supporting the learner to think independently and solve problems. Additionally, if direct access to the AI-chat is provided, learners may get the correct answer without effort, which could lead to academic dishonesty and weaken their critical thinking.

TABLE 3. Example of prompt templates for the Code Correctness Check Module (CCM).

Strict code check prompt	
Instruction	Analyze the [Submitted code] provided by the student for the [PythonProblem]. Although it passed all test cases, ensure rigorous validation beyond the test cases. If the [SubmittedCode] matches any of the following criteria, respond only with the corresponding criteria items' name. If not, respond with "No issues found" Unnecessary Code: Code contains elements not essential for solving the problem. Requirement not met: The solution ignores a specified requirement and solves the problem differently. Hard Coding: The solution uses hard-coded input/output examples instead of general logic. Computation Error: Errors occur due to overly complex or incorrect logic.
Exercise	[Instruction] Function sum() receives three integers and returns the total. Function avg() returns the average.
Test	Input: 10, 20, 30; Output: 60, 20.0
Submitted Code	<pre>def sum(a,b,c): print(a+b+c) def avg(a,b,c): print((a+b+c)/3) a = int(input()) b = int(input()) c = int(input()) sum(a,b,c) avg(a,b,c)</pre>
Solution	<pre>def Sum(a, b, c): s = a+b+c return s def avg(a, b, c): s = Sum(a, b, c) avg = s/3 return avg a = int(input()) b = int(input()) c = int(input()) print(Sum(a, b, c)) print(avg(a, b, c))</pre>

To prevent this issue, learners can only receive feedback on their submitted code by clicking the “Ask Code Tutor” button. This approach also addresses concerns that automatically generating detailed feedback on every code submission may feel overwhelming to learners [26], by allowing them to receive structured feedback only when they choose to, without requiring prompt engineering.

Fourth, an integrated UI must be provided. Presenting problems, writing code, and checking code review comments should be provided on one page to enhance the learning and ease of use.

Fifth, an editor for Python programming must be provided. The Python style was not applied to the code editor area of the existing judge system, so that reducing the readability of the code. To solve this, an instance of Monaco Editor for Python programming was added. In this editor, learners can program in Python with improved readability and ease of writing.

D. SYSTEM DEVELOPMENT

The system was progressively improved through three rounds of usability testing. The following describes the final version of the system used in the final round of testing with primary and secondary school students.

1) SYSTEM FLOW

Learners can request either an answer check or a code review for their submitted code. As shown in Figure 3, the Code

Validation Module (CVM) first checks whether the code is a valid Python program. If valid, the Code Correctness Check Module (CCM) uses traditional test case methods to evaluate correctness and provides additional feedback on code quality. For code review requests, the CVM validates the code, and if it passes, the Code Review Module (CRM) determines whether a review is necessary. If required, the CRM generates feedback. All user activity is logged and stored for usage analysis.

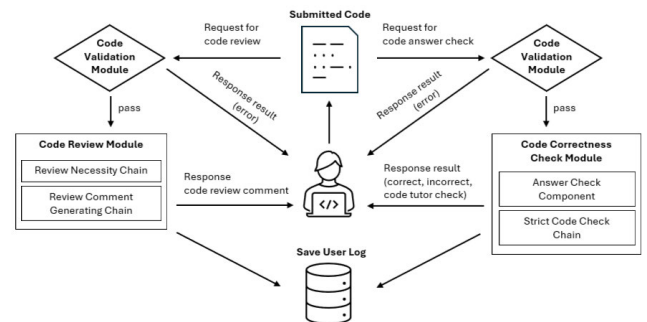


FIGURE 3. Overall system flow for code submission, validation, review, and logging.

2) SYSTEM CONFIGURATION AND ENVIRONMENT

In order to quickly develop this system as a web application and minimize deployment workload, the backend and frontend were built jointly using the Next.js framework. The build and deployment of the system were carried out using Microsoft Azure's Static Web App service, as illustrated in Figure 4.

Additionally, LangChain was adopted to systematically manage prompt templates and prevent issues where the GPT output may deviate from the intended format, as noted in prior research [12].

Front-end	Web	Next.js		ANTD	
	Server	Microsoft Azure			
Back-end	API / DB	Node.js		MongoDB	
	Code Check / Review Module	GPT		LangChain	

FIGURE 4. System configuration and environment.

3) USER INTERFACE AND EXECUTION EXAMPLES

Figure 5 shows the UI of the developed system, and the roles and functions of each part are as follows:

- 1) This area shows the details of the selected problem, including the exercise requirements, input examples, output examples, and constraints.
- 2) This is a Python editor for writing code. When a user clicks “Ask Code Tutor”, the annotation “Code to fix” is added to the part of the submitted code that needs to be modified, as shown in Figure 6.
- 3) This button submits the learner’s code, and the GPT-4o-based module validates test cases and code quality, displaying results in a pop-up, as shown in Figure 7.
- 4) This button requests code review comments through the GPT-4o endpoint using the CRM.
- 5) This area shows the feedback received from GPT-4o, which can be seen in Figure 8.

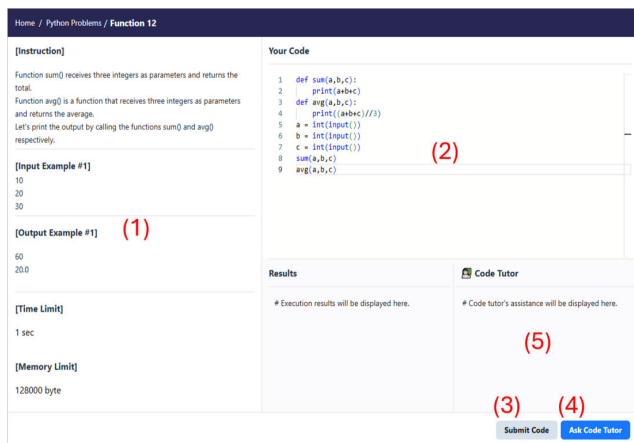


FIGURE 5. Main user interface for AI assistance.

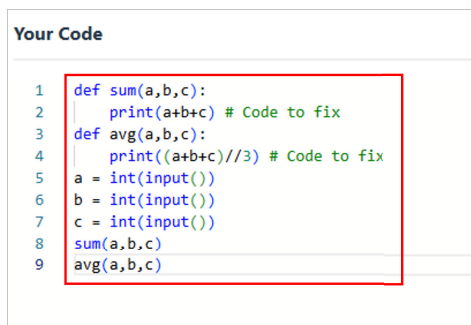


FIGURE 6. Example of line highlighting in submitted code.

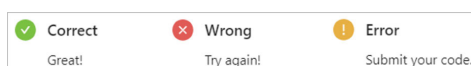


FIGURE 7. Example of the three states for answer validation.

4) CODE CORRECTNESS CHECK MODULE (CCM)

The online judge provides relatively consistent answer checking but has limitations in evaluating code quality and analyzing error types. As shown in Figure 9, the Code

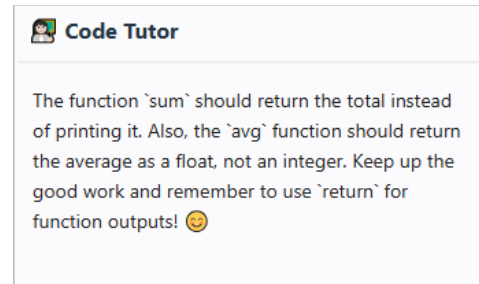


FIGURE 8. Example of displayed code review comments.

Correctness Check Module (CCM) is designed to go beyond simple test case-based validation and enable stricter code verification. This module employs a collaborative validation flow that integrates both compilation and answer checking, allowing it to detect issues in code quality that cannot be identified through mechanical validation alone.

The Answer Check Component primarily evaluates incorrect output, syntax errors, and code complexity using server-side resources. If the submitted code passes all test cases, additional verification is performed through the Strict Code Check Chain (SCC). The SCC analyzes the code and provides feedback based on the following four criteria. If the code passes all checks, the system returns a “No issues found” message and stops the evaluation.

- **Unnecessary code:** Includes code that is irrelevant to solving the problem.
- **Requirement not met:** Solves the problem in a way that deviates from the given constraints.
- **Hard coding:** Uses fixed values taken directly from the sample output, rather than that general logic.
- **Computation error:** Contains unnecessarily complex calculations or logical flaws.

5) CODE REVIEW MODULE (CRM)

An analysis of user activity logs from the usability test revealed an issue where the model generated unnecessary code reviews by overanalyzing the learner’s intent, even when the submitted code was too simple or meaningless. For example, review feedback were automatically generated for code such as `print()` or simple numeric inputs. Such unnecessary code reviews result in excessive token consumption in large-scale educational environments, leading to significant costs.

To address this issue, the code Review Module (CRM) was enhanced to prevent unnecessary token generation. As shown in Figure 10, CRM has been enhanced to prevent unnecessary token generations. The Review Necessity Chain (RNC) determines whether a code review is required based on the following three criteria.

- **Yes:** The submitted code contains errors or can be improved.
- **No correct:** The submitted code is already correct and matches the expected solution.

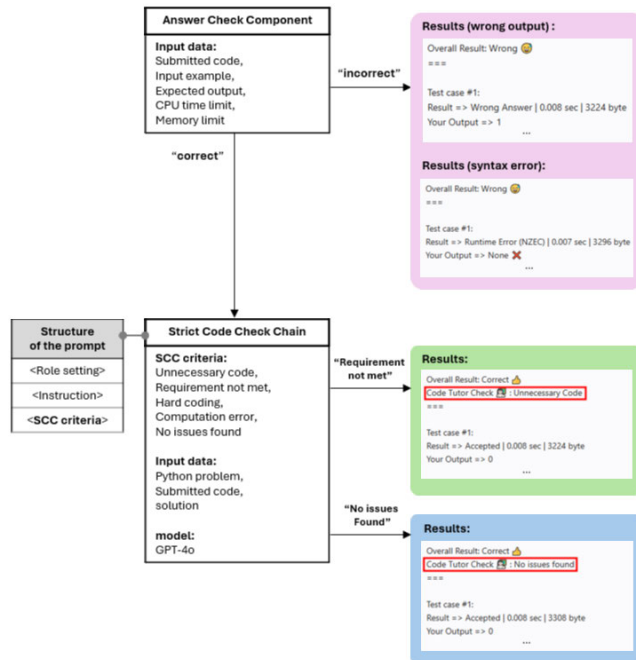


FIGURE 9. Workflow of the Code Correctness Check Module (CCM) for answer and strict code correctness checking.

- **No meaningless:** The submitted code is too simple and does not represent a valid attempt to solve the given problem.

If the RNC determines that a review is necessary, the Review Generation Chain generates a reviewed version of the learner's code by highlighting lines that require improvement, along with corresponding comments.

6) REVIEW NECESSITY CHAIN (RNC) LOGIC

The Review Necessity Chain (RNC) is a subcomponent of the Code Review Module, positioned before the Review Comment Generation Chain in the workflow shown in Figure 10.

As part of this process, the RNC performs a filtering step before invoking the review comment generation module. The RNC receives three inputs: the problem description, the learner's submitted code, and the instructor-defined reference solution. It outputs a single label, denoted as `reviewLabel`, which indicates whether the code requires review.

It returns a single label—`reviewLabel`—which is one of `yes`, `no_correct`, or `no_meaningless`. If the code contains mistakes or can be improved, the label is set to `yes`. If the code already matches the solution, it returns `no_correct`. If the code is too simple or contextually irrelevant (e.g., a lone `print()` statement), the system returns `no_meaningless`. This design avoids unnecessary GPT API calls while still providing effective

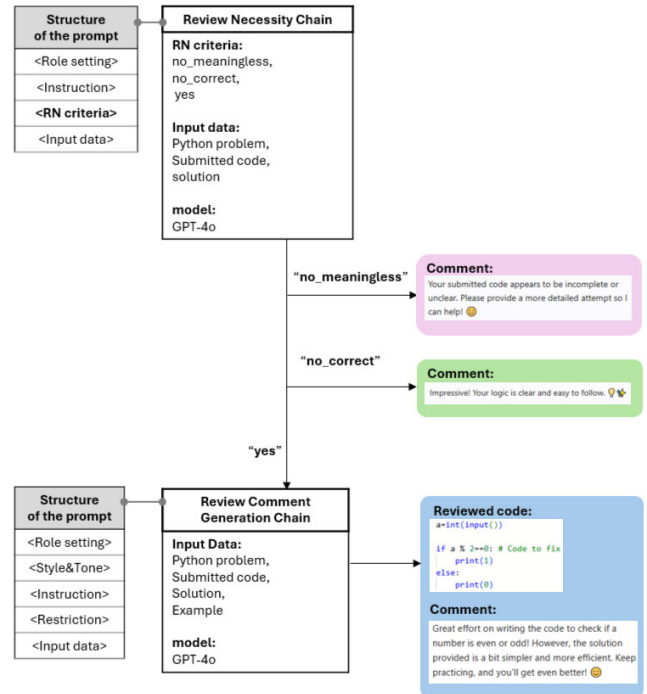


FIGURE 10. Workflow of the Code Review Module (CRM) for filtering unnecessary reviews and generating feedback.

and context-appropriate guidance to learners. This process is illustrated in pseudocode form in Algorithm 1.

Depending on the output label, the system takes different follow-up actions. If the label is `no_meaningless`, a predefined system message is displayed to the user, indicating that the submitted code appears incomplete and requesting a more detailed attempt. If the label is `no_correct`, the system selects a predefined congratulatory message at random and displays it to the user.

E. USABILITY TEST AND IMPROVEMENT

In order to verify the suggested system, six software education experts participated in two rounds of usability testing. All experts had at least two years of programming education experience. Qualitative feedback was collected through post-use interviews based on service elements for AI-based web applications proposed by Moon Hee-Jeoung [29], including user experience, support, and automation. Based on the feedback gathered from these expert user evaluations, an improvement strategy for the system was developed, and key areas for enhancement—particularly related to the code review functionality—were prioritized. The summarized results of this strategy are presented in Table 4. All strategies in Table 4 were implemented to improve the final system, which was evaluated through a third usability test with K-12 learners using a 7-point Likert-scale questionnaire adapted from the expert evaluation.

Algorithm 1 Review Necessity Chain (RNC) Logic

Input: Python Problem: problem description,
Submitted Code: student's code,
Solution: instructor's answer code

Output: reviewLabel $\in$ {yes, no_correct, no_meaningless}

Procedure:

Step 1: Initialize model with:
model = GPT-4o, temperature = 0.2,
max_tokens = 50, top_p = 0.1

Step 2: Define promptTemplate with:
- System role: "You are a code reviewer who evaluates whether a review is necessary."
- User instruction:
Respond 'no_meaningless' if code is trivial (e.g., print()).
Respond 'no_correct' if code is correct.
Respond 'yes' if code has mistakes.

Step 3: Create chain by combining promptTemplate, model, and outputParser, then invoke it with pythonProblem, submittedCode, and solution.

Step 4: Receive the model's response and return it as reviewLabel

Due to the involvement of underage participants, the usability test followed a structured process. First, participants submitted a signed consent form and received a link to the online system. They were given the option to complete the test either at home or at the DLAB campus under supervision. Second, after accessing the system, participants signed up, logged in, and solved Python programming problems using the system. During this process, they received strict answer validation and real-time code review feedback. Finally, participants submitted their feedback via an online questionnaire.

The participants were recruited with approval from the DLAB campus director. A total of 11 K-12 learners participated in the final usability test.

IV. SYSTEM EVALUATION

The evaluation focuses on measuring the accuracy of error detection, the semantic validity of AI-generated feedback, the efficiency of the code review pipeline, and the educational impact in large-scale classrooms.

Specifically, the first aspect involves examining whether the Code Correctness Check Module (CCM) improves the relative error detection rate compared to a traditional online judge system. The second aspect evaluates the semantic similarity between AI-generated feedback and that produced by human instructors. The third aspect concerns how effectively the pipeline reduces unnecessary reviews while preserving essential feedback. Lastly, the fourth aspect assesses the system's impact on learning efficiency and its potential to reduce teachers' feedback workload in large-scale

TABLE 4. Analyzing feedback and improvement strategies.

Element	Improvement strategy	Priority
Function and service description	Adding correct answer code that includes all types of quotes to issues related to the print() function and string formatting	High
	If user submits an error code that does not execute, throw an exception. The evaluation module only asks GPT to check correct and incorrect answers if user submits working code	High
	Account registration and login functions added. Marking solved exercises in the exercise bank	Low
Improved user experience	Add color to instructions, input examples, and output examples	Low
	Developing responsive UI design suitable for monitor resolution	Low
Troubleshooting and support	If user submits an error code that does not execute, throw an exception. The evaluation module only asks GPT to check correct and incorrect answers if user submits working code	High
	Adjusting to the optimal maxTokens value and limiting the maximum number of sentences in code review comments	High
	Adding strict constraints on style and tone to the code review model and adjusting TopP and Temperature values to limit different response outcomes	High
	Refining logic related to slicing responses from GPT	High
Automated process	Adding a user guide	Medium

educational settings. These aspects are evaluated in detail in the following subsections.

A. RQ1: STRICT CODE CORRECTNESS CHECK

To evaluate whether the proposed Code Correctness Check Module (CCM) improves the relative error detection rate over the traditional online judge (OJ), the performance of both systems was compared based on four common error types frequently observed in learner-submitted code, as identified from the OJ system logs. These error types were selected because they are often overlooked by traditional online judges, which rely on input-output comparison.

It is hypothesized that CCM achieves a higher relative error detection rate than the traditional OJ, particularly in identifying subtle issues such as unmet requirements and unnecessary code.

The four error types considered in this study are as follows:

- **Unnecessary code:** Non-essential code added.
- **Requirement not met:** Submitted code ignores specified requirements.
- **Hard coding:** Hardcoded solutions based on input/output examples.
- **Computation error:** Calculation errors from complex logic.

To construct the test set, 26 programming problems with a correct answer rate of 50% or lower, resulting in 72 representative learner-submitted codes that were successfully analyzed using the system. To quantify improvement in error detection, the relative error detection rate (REDR) was computed using the following formula:

$$\text{REDR} = \frac{S_f - S_{oj}}{S_{oj}} \times 100$$

where S_f is the number of errors detected by the system, and S_{oj} is the number of errors detected by the OJ.

As shown in Table 5, the system detected errors more effectively than the OJ across all error types. Notably, the REDR for the “Requirement not met” category reached 42.86%, demonstrating that the system evaluates code requirements more rigorously than the OJ. Although the REDR for “Hard coding” was relatively low (18.18%), this is likely due to the OJ already possessing some capability to detect hardcoded patterns.

Overall, the system outperformed the OJ in detecting all types of errors, with particularly significant improvement in identifying “Requirement not met” errors. These findings suggest that the proposed system not only offers more comprehensive code quality evaluation but also possesses enhanced capability to detect logical flaws that are often overlooked by OJ.

To examine whether these performance differences are statistically significant, Fisher’s exact test was applied to each error type. Statistically significant differences in detection success between the proposed system and the OJ were observed in three of the four error categories—unnecessary code, requirement not met, and hard coding—with all corresponding p-values falling below the 0.05 threshold.

In contrast, the computation error category did not yield a statistically significant difference between the two systems ($p = 0.3870$), indicating that both systems may share similar limitations in detecting arithmetic faults. This similarity likely stems from both systems’ reliance on output-based evaluation, which fails to capture deeper logical errors. These findings highlight the need for targeted improvements in the current system’s ability to detect complex computational issues.

Overall, these findings demonstrate that the Code Correctness Check Module improves the relative error detection rate compared to the baseline online judge in three of the four error categories. RQ1 can thus be answered in the affirmative, supported by both performance metrics and statistical significance.

TABLE 5. Relative error detection rate (REDR) and statistical significance across error types.

Error type	REDR (%)	p-value (Fisher’s exact test)
Unnecessary code	31.58	$p < 0.0001$
Requirement not met	42.86	0.0031
Hard coding	18.18	$p < 0.0001$
Computation error	33.33	0.3870

B. RQ2: SEMANTIC SIMILARITY WITH HUMAN FEEDBACK

In order to determine whether the system-generated feedback is comparable to expert-written reviews, the degree of semantic similarity between feedback generated by the Code Review Module (CRM) and that of human instructors was examined. To support this evaluation, BERTScore was employed as the semantic similarity metric.

A total of 27 test cases were prepared as evaluation tasks. To establish a reliable reference set, three programming education experts—each with over three years of teaching experience—were recruited and asked to write code review feedback for all 27 test cases, following a standardized protocol.

To ensure consistency and fair comparison, the following review guidelines were applied: 1) Provide immediate and accurate feedback while maintaining a supportive tone. 2) Annotate any lines requiring modification with a “Code to fix” comment.

Subsequently, the CRM was used to generate code reviews for the same 27 test cases. The system-generated reviews were then compared with the expert-written references using BERTScore to quantitatively assess their semantic similarity.

As shown in Table 6 and Figure 11, BERTScores across the 27 test cases ranged from 0.6578 to 0.9045, with a mean of 0.8001 and a standard deviation of 0.0729.

To validate the reliability of the observed semantic similarity, a 95% confidence interval for the mean BERTScore was computed based on the t -distribution. The resulting interval of 0.8001 ± 0.0288 , corresponding to the range [0.7713, 0.8289], confirms that the reported performance is statistically robust.

These results—reflecting both a high average semantic similarity and its statistical reliability—indicate that, on average, CRM produced feedback that was over 80% semantically similar to that of human instructors, demonstrating both high quality and consistency in generation.

Accordingly, the CRM demonstrated alignment with expert-written feedback, supporting a positive answer to RQ2.

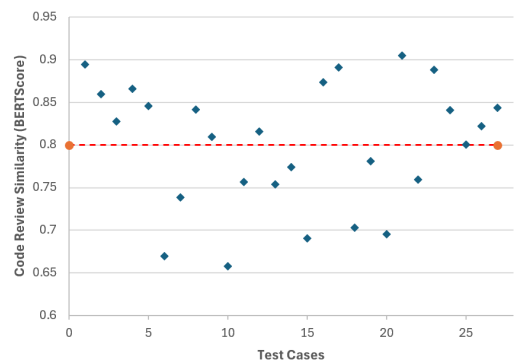


FIGURE 11. Distribution of semantic similarity between the Code Review Module (CRM) generated and human instructor code reviews (BERTScore).

C. RQ3: REVIEW NECESSITY FILTERING

It is assumed that the Code Review Module (CRM)’s pipeline can effectively suppress unnecessary reviews—particularly for trivial or meaningless code—while preserving essential instructional feedback.

To evaluate this assumption, the experiment was conducted under a simulated classroom deployment scenario,

TABLE 6. Semantic similarity between the Code Review Module (CRM) and human instructor code reviews (BERTScore).

Test case	Exercise ID	Instructor review len.	System review len.	BERTScore (F1)
1	1173	235	257	0.8944
2	1178	242	324	0.8594
3	1213	148	223	0.8276
4	1221	201	303	0.8657
5	1222	230	279	0.8455
6	1291	183	222	0.6697
7	1293	328	281	0.7386
8	1334	329	371	0.8414
9	1335	247	277	0.8094
10	1376	328	347	0.6578
11	1382	250	274	0.7568
12	1385	256	304	0.8156
13	1400	231	267	0.7540
14	1401	275	298	0.7737
15	1422	349	220	0.6906
16	1430	267	281	0.8735
17	1460	388	380	0.8910
18	1461	226	227	0.7031
19	1462	365	264	0.7811
20	1465	166	247	0.6952
21	1466	319	395	0.9045
22	1467	230	258	0.7591
23	1470	229	302	0.8880
24	1187	264	345	0.8406
25	1189	173	299	0.8002
26	1226	329	303	0.8219
27	1229	386	381	0.8434

comparing the CRM with leading state-of-the-art (SOTA) code review systems.

This comparison was intended to assess which system provides a more practical balance between efficiency and review quality, especially in educational contexts where minimizing unnecessary feedback and token usage is important.

First, user activity logs from a usability test conducted in March were used to construct a dataset of 100 test cases: 50 containing trivial or meaningless code unlikely to require feedback, and 50 containing cases with valid or missing reviews.

Second, to ensure a rigorously fair comparison, all models were evaluated under an identical and controlled prompt environment, designed to emulate a three-stage review process: 1) code validation, 2) review necessity prediction, and 3) review comment generation.

To align the conditions, the prompt templates used in the CRM pipeline were directly applied to all comparison systems. Since most SOTA systems are accessed via application-level interfaces without parameter configuration, the prompts were input through their standard prompt fields to replicate the review conditions as closely as possible.

However, the suggested system includes the Code Validation Module, built with regular expressions code to detect empty or comment-only code, which cannot be achieved through prompting. To address this discrepancy, additional prompt instructions were introduced to guide LLMs in performing equivalent validation tasks.

As shown in Table 7, the CRM achieved the highest blocking rate for unnecessary feedback, reaching 86%. Furthermore, as presented in Table 8, the review omission rate remained at 6%, which is comparable to other SOTA systems.

These findings demonstrate that the CRM pipeline effectively filters out non-essential reviews without reducing feedback quality, highlighting its potential for scalable use in educational settings.

In summary, the results validate the assumption in RQ3 by confirming that the pipeline reduces unnecessary reviews while retaining essential instructional feedback.

TABLE 7. Blocking performance of code review systems for meaningless code.

System	Successful blocking	Failed blocking	Blocking success rate (%)
Code Review Module	43	7	86
Copilot-GPT4o	34	16	68
Copilot-Claude 3.5	26	24	52
Amazon Q	16	34	32

TABLE 8. Review omission rates of code review systems for required code reviews.

System	Missed reviews	Successful reviews	Review omission rate (%)
Code Review Module	6	44	12
Copilot-GPT4o	4	46	8
Copilot-Claude 3.5	1	49	2
Amazon Q	6	44	12

D. RQ4: EDUCATIONAL IMPACT & TEACHER WORKLOAD REDUCTION

This study investigates whether the proposed code review system can enhance students' learning efficiency by providing timely and supportive feedback, while also reducing instructors' workload in large-scale classroom environments.

To verify the effectiveness of the proposed system through usability testing, an experiment was conducted at an educational institution in Seoul, involving 11 primary and secondary school students during February 2025.

As shown in Table 9 and Figure 12, Students rated the supportive tone with an average score of 6.64, indicating that they perceived the system as a friendly and encouraging learning tool. The next highest score, 6.27, was given to the precision of marking error lines, reflecting students' satisfaction with the system's ability to clearly identify erroneous lines in their code. Interview responses supported these findings, with comments such as: "Learning to code is enjoyable with just the code tutor," "Thanks to the kind explanations and quick feedback, I can study without a teacher," and "It helped me develop habits for code optimization and debugging, and learn the right attitude for problem solving."

On the other hand, the lowest score was given to usefulness-learning Independence, with an average of 4.82. Students expressed concerns that relying solely on AI

feedback may have limitations compared to learning with a teacher. Interview feedback included suggestions such as: “Adaptive difficulty and differentiated feedback should be provided,” and “It would be helpful if the AI adjusted its feedback style based on the learner’s proficiency level.”

This usability test expanded the scope of evaluation beyond educational experts to include primary and secondary school students, providing broader insights into the AI code review system’s educational impact and usability.

The results suggest that the proposed system has the potential to function as an educational tool that enhances conceptual understanding and coding skills, supports self-directed learning through prompt feedback, reduces teachers’ workload, and contributes to scalability in large classroom environments.

Collectively, these findings confirm that the system meets the dual objectives of enhancing learning efficiency and alleviating teacher burden. This provides empirical support for a positive answer to RQ4.

TABLE 9. Summary of student survey results on code review.

Criteria	Survey Item	Mean
Precision	Code tutor accurately pointed out errors in the code and suggested solutions.	5.73 (1.79)
	Code tutor added comments to the exact lines of code that needed modification.	6.27 (0.90)
Usefulness	The improvements suggested in code tutor’s comments actually help solve the problem.	5.73 (1.90)
	Code tutor’s comments alone is sufficient to solve the problem.	4.82 (1.66)
Specificity	The code tutor’s comments are not general but point out specific errors or problems.	5.27 (1.79)
Supportive tone	The code tutor’s comments provide a kind and encouraging impression.	6.64 (0.50)
Learning effect	Code tutor comments can help learners understand new concepts or improve skills.	5.82 (1.54)

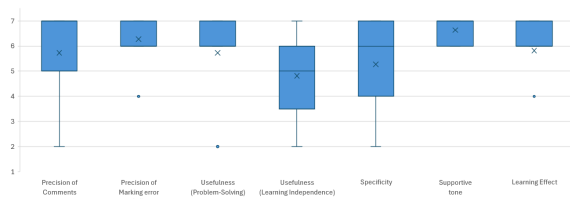


FIGURE 12. Distribution of student responses in the code review survey.

V. CONCLUSION AND FURTHER STUDY

The system has been verified as an effective and sustainable programming language learning tool for primary and secondary school students through a series of research questions.

First, the Code Review Module (CRM) demonstrated improved error detection capabilities compared to the online judge, particularly in identifying requirement violations and computation errors. The relative error detection rate showed statistically significant improvements in the detection of three major error categories—namely, Unnecessary code, Requirement not met, and Hard coding—thereby demonstrating the

improved accuracy and reliability of the system’s evaluation process.

Second, confirmed that the feedback generated by the CRM was semantically aligned with instructor-written reviews. Based on BERTScore evaluation, the system’s average similarity score exceeded 0.80, indicating that AI-generated feedback can replicate the tone and accuracy of human instructors.

Third, the result of demonstrated that the system’s review pipeline successfully reduced unnecessary feedback generation. It achieved the highest blocking rate of meaningless code reviews (86%) among SOTA systems while maintaining a low omission rate of essential reviews (6%).

Finally, student feedback confirmed the educational value of the system. Learners rated the system highly for its supportive tone and precision in marking errors. Interviews further revealed that students felt empowered to engage in self-directed learning. However, some noted limitations in terms of adaptive difficulty and personalized feedback.

In summary, the results of the four research questions indicate that the system offers stricter code evaluation, maintains review quality, filters unnecessary feedback, and provides pedagogical benefits in large-scale educational settings.

However, the final usability test involved a relatively small sample size of 11 student participants, which may limit the generalizability of findings. Future studies will aim to include a larger number of students across specific grade levels and conduct evaluations in real classroom settings to more rigorously assess the system’s adaptability. While restricting AI-generated feedback, especially for meaningless or overly simplistic code, can reduce overreliance in introductory programming courses, it may also limit students’ long-term opportunities to explore and develop responsible use of generative tools essential for future learning environments. The system is accessible at: <https://www.codetutor119judge.com/>

ETHICAL CONSIDERATIONS

This study involved underage participants (elementary and middle school students) and was conducted with explicit parental consent and organizational approval. The usability testing was carried out with permission from the director of DLAB Daechi Campus and company leadership. Participant recruitment was conducted via printed notices at the institution, and a total of 11 students were involved over the period of February 6-13, 2025. All personal data, including usage logs and survey responses, were anonymized and stored securely. The data were used solely for research purposes to evaluate the effectiveness and usability of the proposed system.

Before participating in the usability test, all students and their legal guardians were informed about the purpose and procedures of the usability testing. Parental consent forms were collected in advance to ensure compliance with ethical standards involving underage participants. Only students whose parental consent forms had been submitted were

allowed to participate in the study. These participants engaged in Python programming tasks using the proposed system and subsequently completed a post-test survey to evaluate their experience.

REFERENCES

- [1] M. of Science and N. I.S. A. ICT. (2021). *Global Digital Competency Education*. Accessed: Dec. 2023. [Online]. Available: <https://www.nia.or.kr/cnt/edu>
- [2] *OECD Skills Outlook 2019: Thriving in a Digital World*, O. for Economic Co-operation and Development, OECD, Paris, France, May 2019.
- [3] M. Cardona, R. Rodriguez, and K. Ishmael. (May 2025). *Artificial Intelligence and the Future of Teaching and Learning: Insights and Recommendations*. Office of Educational Technology, U.S. Department of Education. Accessed: May 2025. [Online]. Available: <https://www.ed.gov/sites/ed/files/documents/ai-report/ai-report>
- [4] U.S. Department of Education. (2025). *Artificial Intelligence (AI) Guidance and Use Case Inventory*. Accessed: May 3, 2025. [Online]. Available: <https://www.ed.gov/about/ed-overview/artificial-intelligence-ai-guidance>
- [5] Ministry of Education and Culture Helsinki. (2023). *Policies for the Digitalisation of Education and Training Until 2027*. Accessed: May 2025. [Online]. Available: <http://urn.fi/URN:ISBN:978-952-263-945-5>
- [6] Ministry of Education Singapore. (2024). *Transforming Education Through Technology: Masterplan 2030*. Accessed: May 3, 2025. [Online]. Available: <https://www.moe.gov.sg/education-in-sg/educational-technology-journey/edtech-masterplan>
- [7] M. of Education. (2024). *Exploring the 2022 Revised Elementary School Curriculum Organization and Operation*. Accessed: Dec. 2023. [Online]. Available: https://www.gne.go.kr/upload_data
- [8] British Informatics Olympiad. (2025). *Rules of the 2025 British Informatics Olympiad*. Accessed: May 2025. [Online]. Available: <https://www.olympiad.org.uk/2025/rules.html>
- [9] College Board. (2023). *Ap Computer Science a—The Exam*. AP Central. Accessed: May 2025. [Online]. Available: <https://apcentral.collegeboard.org/courses/ap-computer-science-a/exam>
- [10] S. Choi, D. Lee, J. Kim, Y. Jang, and H. Kim, “Designing llm-based code reviewing learning environment for programming education,” *J. Korean Assoc. Comput. Educ.*, vol. 26, no. 5, pp. 1–11, Jul. 2023.
- [11] M. Kazemitabaar, R. Ye, X. Wang, A. Z. Henley, P. Denny, M. Craig, and T. Grossman, “CodeAid: Evaluating a classroom deployment of an LLM-based programming assistant that balances student and educator needs,” in *Proc. CHI Conf. Hum. Factors Comput. Syst.* New York, NY, USA: Association for Computing Machinery, May 2024, pp. 1–20, doi: [10.1145/3613904.3642773](https://doi.org/10.1145/3613904.3642773).
- [12] S. Sahai, U. Z. Ahmed, and B. Leong, “Improving the coverage of GPT for automated feedback on high school programming assignments,” in *Proc. NeurIPS Workshop Generative AI Educ. (GAIED)*, vol. 46. Baton Rouge, LA, USA: MIT Press, 2023, p. 1. [Online]. Available: <https://neurips.cc/virtual/2023/79073>
- [13] M.-F. Wong, S. Guo, C.-N. Hang, S.-W. Ho, and C.-W. Tan, “Natural language generation and understanding of big code for AI-assisted programming: A review,” *Entropy*, vol. 25, no. 6, p. 888, Jun. 2023, doi: [10.3390/e25060888](https://doi.org/10.3390/e25060888).
- [14] H. Nguyen and V. Allan, “Using GPT-4 to provide tiered, formative code feedback,” in *Proc. 55th ACM Tech. Symp. Comput. Sci. Educ.*, Mar. 2024, pp. 958–964. [Online]. Available: <https://dl.acm.org/doi/10.1145/3626252.3630960>
- [15] S. Saylam, N. Duman, Y. Yildirim, and K. Satsevich, “Empowering education with AI: Addressing ethical concerns,” *London J. Social Sci.*, vol. 6, no. 6, pp. 39–48, Sep. 2023.
- [16] M. Jaber, “New innovations in higher education’s academic integrity and classroom strategies,” in *Academic Integrity in the Age of Artificial Intelligence*. Hershey, PA, USA: IGI Global, Feb. 2024, pp. 281–300.
- [17] Y. Xie, S. Wu, and S. Chakravarty, “AI meets AI: Artificial intelligence and academic integrity—A survey on mitigating AI-assisted cheating in computing education,” in *Proc. 24th Annu. Conf. Inf. Technol. Educ.*, Oct. 2023, pp. 79–83.
- [18] B. McDanel and E. Novak, “Designing LLM-resistant programming assignments: Insights and strategies for CS educators,” in *Proc. 56th ACM Tech. Symp. Comput. Sci. Educ.* New York, NY, USA: Association for Computing Machinery, Feb. 2025, pp. 756–762, doi: [10.1145/3641554.3701872](https://doi.org/10.1145/3641554.3701872).
- [19] J. P. Amos, O. A. Amodu, R. A. R. Mahmood, A. B. Abdulqudus, A. F. Zakaria, A. R. Iyanda, U. A. Bukar, and Z. M. Hanapi, “A bibliometric exposition and review on leveraging LLMs for programming education,” *IEEE Access*, vol. 13, pp. 58364–58393, 2025.
- [20] W. Dai, J. Lin, H. Jin, T. Li, Y.-S. Tsai, D. Gasevic, and G. Chen, “Can large language models provide feedback to students? A case study on ChatGPT,” in *Proc. IEEE Int. Conf. Adv. Learn. Technol. (ICALT)*, Jul. 2023, pp. 323–325.
- [21] R. Singh, S. Gulwani, and A. Solar-Lezama, “Automated feedback generation for introductory programming assignments,” in *Proc. 34th ACM SIGPLAN Conf. Program. Lang. Design Implement.* New York, NY, USA: Association for Computing Machinery, Jun. 2013, pp. 15–26, doi: [10.1145/2491956.2462195](https://doi.org/10.1145/2491956.2462195).
- [22] J. Lu, L. Yu, X. Li, L. Yang, and C. Zuo, “LLaMA-reviewer: Advancing code review automation with large language models through parameter-efficient fine-tuning,” 2023, *arXiv:2308.11148*.
- [23] Q. Zhang, T. Zhang, J. Zhai, C. Fang, B. Yu, W. Sun, and Z. Chen, “A critical review of large language model on software engineering: An example from ChatGPT and automated program repair,” 2023, *arXiv:2310.08879*.
- [24] T. Sun, J. He, X. Qiu, and X. Huang, “BERTScore is unfair: On social bias in language model-based metrics for text generation,” in *Proc. Conf. Empirical Methods Natural Lang. Process.*, vol. 1, Jan. 2022, pp. 3726–3739s. [Online]. Available: <https://aclanthology.org/2022.emnlp-main.245/>
- [25] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating text generation with BERT,” 2019, *arXiv:1904.09675*.
- [26] A. S. Crandall, G. Sprint, and B. Fischer, “Generative pre-trained transformer (GPT) models as a code review feedback tool in computer science programs,” *J. Comput. Sci. Colleges*, vol. 39, no. 1, pp. 38–47, Oct. 2023. [Online]. Available: <https://dl.acm.org/doi/10.5555/3636517.3636522>
- [27] A. S. Crandall, B. Fischer, and J. P. Crandall, “WIP: ARTful insights from a pilot study on GPT-based automatic code reviews in undergraduate computer science programs,” in *Proc. IEEE Frontiers Educ. Conf. (FIE)*, Oct. 2024, pp. 1–5. [Online]. Available: <https://ieeexplore.ieee.org/document/10893407>
- [28] C3Coding. (2023). *The Online Judge System for Elementary and Secondary School Students*. Accessed: Dec. 2023. [Online]. Available: <https://judge.c3coding.com/>
- [29] H.-J. Moon, “Evaluation of user satisfaction and effectiveness of web (App)-based artificial intelligence application services,” *J. Digit. Contents Soc.*, vol. 24, no. 10, pp. 2611–2617, Oct. 2023.



DONG-KYU LEE received the B.S. degree in software engineering from the Cyber University of Korea, in 2022, and the M.S. degree in computer engineering from Hanyang University, Seoul, South Korea, in 2024, where he is currently pursuing the Ph.D. degree with the Division of Computer Science and Engineering.

His research interests include AI agent systems, convergence of artificial intelligence in the education domain, and software system design and development.



INWHEE JOE received the B.S. and M.S. degrees in electronics engineering from Hanyang University, Seoul, South Korea, and the Ph.D. degree in electrical and computer engineering from Georgia Institute of Technology, Atlanta, GA, USA, in 1998.

Since 2002, he has been with the Division of Computer Science and Engineering, Hanyang University, where he is currently a Professor. His research interests include the Internet of Things (IoT), cellular systems, wireless power communication networks, embedded systems, machine learning/deep learning, computer vision, and performance evaluation.

...